

Chunking and Retrieval

Why RAG fails, and how to fix it · video 2:39

This lesson answers one question: why does RAG fail on real documents, and how do you fix it? Long documents must be cut into **chunks** before retrieval, and the way you cut decides whether the right fact is found. In a measured test, **whole documents** waste the prompt and silently overflow the embedding model, **fixed-size** chunks cut facts in half, and chunks that follow the text's own **structure** (sections, or sentences with their neighbours) find every answer with few words.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The experiment	▶ 0:08-0:41	2	☐
2. Whole documents: found, but wasteful and truncated	▶ 0:41-1:05	2	☐
3. Fixed-size chunks cut facts in half	▶ 1:05-1:26	2	☐
4. Cut along the text's own structure	▶ 1:26-2:11	2	☐
5. Measure what works	▶ 2:11-2:36	1	☐

Before you start: episodes 2 (context windows), 4 (embedding search) and 5 (RAG) of this series.
Code: `code/p06_chunking` (90 MB model, CPU).

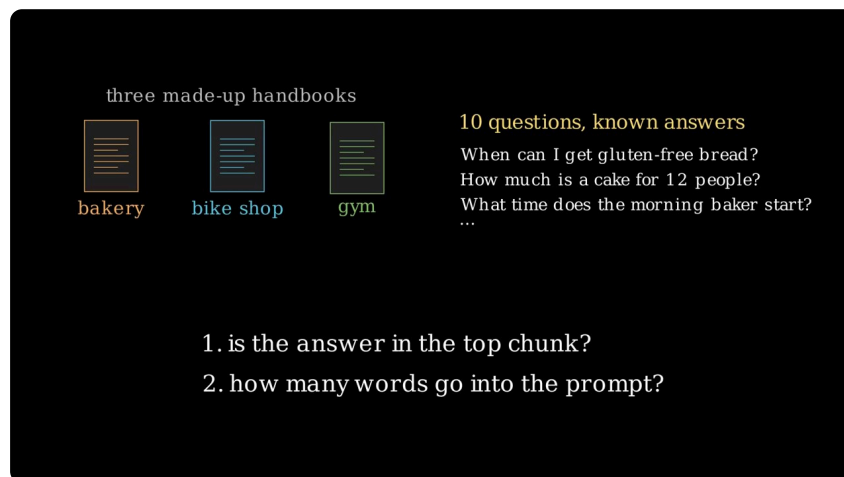
CONCEPT 1 OF 5

The experiment

▶ Video 0:08-0:41

Real documents are long: handbooks, manuals, contracts. You cannot paste a whole library into the prompt, so you **retrieve pieces**, and how you cut those pieces decides whether RAG works at all.

The test: a small library of three made-up handbooks (a bakery, a bike shop, a gym; 467 words) and ten questions about the bakery, each with a known answer. For each way of cutting the text, we check two things: is the answer in the **top chunk**, and how many **words go into the prompt**.



▶ 0:39 Three made-up handbooks, ten questions about the bakery with known answers, and two measurements.

KEY IDEA

Judge a retrieval setup with questions whose answers you know: is the answer retrieved, and at what cost?

Check 1: The experiment

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

Why are the questions "about the bakery" worded carefully, e.g. "Do students get a discount **at the bakery?**"

1.2 TRUE OR FALSE

"Only the first measurement (answer found) matters; the number of words sent is irrelevant."

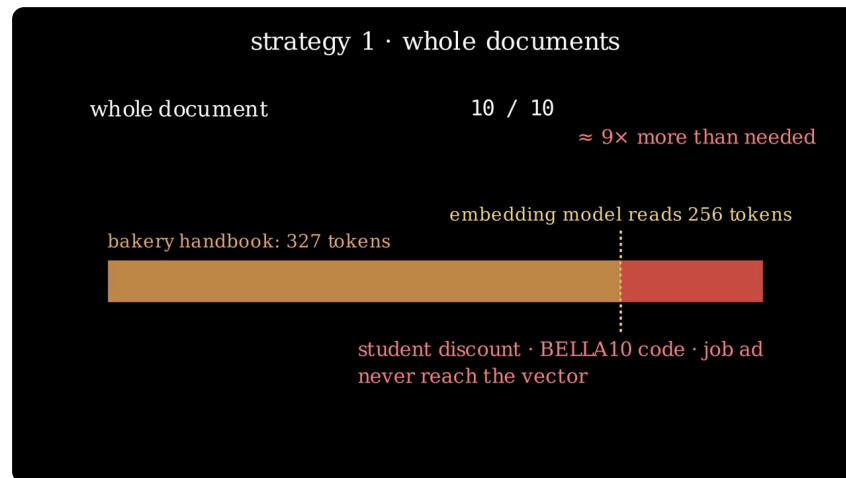
True ☐ False ☐

CONCEPT 2 OF 5

Whole documents: found, but wasteful and truncated

▶ Video 0:41-1:05

With no cutting (one vector per handbook) retrieval found the right handbook every time: 10 / 10. But it sends all **255 words** for every question, about nine times more than needed.



▶ 1:04 The bakery handbook is 327 tokens; the embedding model reads 256. The end never reaches the vector.

And there is a hidden problem: this embedding model reads at most **256 tokens**. The bakery handbook is **327**. Everything after “we accept cash, cards and” (the student discount, the BELLA10 code, the job ad) never makes it into the vector, and nothing warns you.

KEY IDEA

Embedding models have their own input limit. Text beyond it is silently ignored, so long chunks hide their endings.

Check 2: Whole documents: found, but wasteful and truncated

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

The handbook is 327 tokens and the model reads 256. What fraction of the handbook is ignored, to the nearest percent?

2.2 MULTIPLE CHOICE

Whole-document retrieval still scored 10 / 10, even for questions about the ignored end. Why?

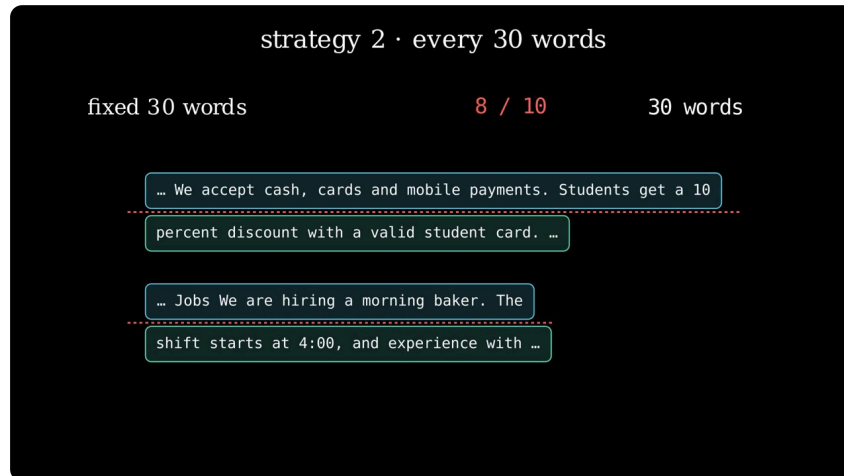
- A. The ignored text is read later by the LLM
- B. The rest of the handbook is clearly about the bakery, so the bakery document still wins against the bike shop and the gym
- C. The model memorised the handbook
- D. Truncation never happens with sentence models

CONCEPT 3 OF 5

Fixed-size chunks cut facts in half

▶ Video 1:05–1:26

Cutting every **30 words**, wherever the cut falls, gives small chunks but only **8 / 10**. Look at the cuts: one chunk ends “*Students get a 10*” and the word *percent* is in the next chunk; another ends “*We are hiring a morning baker. The*” and the start time, 4:00, is in the next one. The facts were cut in half.



▶ 1:24 Real 30-word cuts: “Students get a 10 | percent discount” and “a morning baker. The | shift starts at 4:00”.

KEY IDEA

Cuts that ignore the text's structure separate a fact from the words that make it findable.

Check 3: Fixed-size chunks cut facts in half

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

For “What time does the morning baker start?”, which chunk did fixed 30-word chunking rank first, and why is that a failure?

3.2 TRUE OR FALSE

“Smaller fixed chunks are always more precise, so 15 words beats 30.”

True ☐ False ☐

CONCEPT 4 OF 5

Cut along the text's own structure

▶ Video 1:26-2:11

Cutting along the text's own structure works: **one chunk per section**, with its heading, finds **10 / 10** with about **38 words** each; **each sentence with its neighbours**, so the chunks overlap, finds **10 / 10** with **28 words**.

cut along the text's own structure

whole document	10 / 10	255 words
fixed 30 words	8 / 10	30 words
one per section	10 / 10	38 words
sentence + neighbours	10 / 10	28 words



each sentence + its neighbours: chunks overlap

▶ 1:39 One chunk per section: 10 / 10 with 38 words. Sentence + neighbours: 10 / 10 with 28 words.

other ways retrieval fails

- different words than the document
→ rewrite the question · add keyword search
- the answer needs two chunks
→ retrieve more chunks
- too much retrieved
→ the key fact gets lost in the middle (ep. 2)

▶ 2:10 Other failures and their fixes.

So: cut at **natural boundaries**; keep the **headings**, they carry the topic; **overlap** a little, so no fact falls between two chunks; size chunks to hold **one answer**. And remember the other failures: the question may use different words than the document (rewrite it, or add keyword search); the answer may need two chunks (retrieve more); retrieve too much and the key fact gets lost in the middle.

KEY IDEA

Good chunks are self-contained: one topic, its heading, and enough neighbouring text that no fact is split.

Check 4: Cut along the text's own structure

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Order the four strategies from most to fewest words sent per question: *sentence + neighbours* · *whole document* · *one per section* · *fixed 30 words*.

4.2 MULTIPLE CHOICE

A user asks “refund rules?” but the policy says “returns are accepted within 14 days”. Which fix targets this failure?

- A. Bigger chunks
- B. Rewrite the question (or add keyword search with synonyms)
- C. Fewer chunks
- D. A higher temperature

CONCEPT 5 OF 5

Measure what works

▶ Video 2:11-2:36

In code, the good chunkers are short: split on **blank lines** for sections, or split into **sentences** and join each with its **neighbours**.

```
1 def by_section(text):
2     return [s.strip() for s in text.split("\n\n")] # blank lines
3
4 def sentence_window(text):
5     s = sentences(text)
6     return [" ".join(s[max(0, i - 1):i + 2]) for i in range(len(s))] # + neighbours
7
```

▶ 2:19 The two good chunkers are a few lines each.

Most of all, **measure**: write a few questions with known answers, check what retrieval returns, change one thing at a time, and keep what works.

KEY IDEA

Chunking choices are empirical: a ten-question test catches problems that are invisible by inspection.

Check 5: Measure what works

Answer all 1 questions before looking at the answers. Pass mark: all correct.

5.1 HANDS-ON CODE

Try it yourself. Run this at the end of `chunking.py`. (a) What scores do 60-word chunks, 15-word chunks and single sentences (no neighbours) get? (b) Which question do single sentences miss, and why do neighbours fix it?

```
for label, fn in [("fixed 60", lambda t: fixed(t, 60)),
                 ("fixed 15", lambda t: fixed(t, 15)),
                 ("single sentences", sentences_of)]:
    chunks = chunk_library(fn)
    print(label, evaluate(chunks, show_misses=True))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The experiment · rewatch ▶ 0:08-0:41

1.1 The gym also has student prices; without “at the bakery” the question is ambiguous and a gym chunk would be a fair answer. A test only measures retrieval if each question has one right answer.

1.2 False. Every extra word costs time and money and can bury the key fact (episode 2).

2. Whole documents: found, but wasteful and truncated · rewatch ▶ 0:41-1:05

2.1 22%. $71 / 327 \approx 0.217$.

2.2 B. With only three very different documents, topic alone picks the right one. In a library of many bakery documents it would not.

3. Fixed-size chunks cut facts in half · rewatch ▶ 1:05-1:26

3.1 The bakery's opening-hours chunk. The chunk with “starts at 4:00” lost the words “morning baker”, so it no longer looked like an answer to the question. The question's key words and the answer ended up in different chunks.

3.2 False. In the same test, 15-word chunks found only 4 / 10 answers: more cuts, more facts split.

4. Cut along the text's own structure · rewatch ▶ 1:26-2:11

4.1 whole document (255) → one per section (38) → fixed 30 words (30) → sentence + neighbours (28). Values from the episode's table.

4.2 B. The problem is different words for the same thing, not chunk size.

5. Measure what works · rewatch ▶ 2:11-2:36

5.1 (a) Fixed 60: 9 / 10 (60 words sent); fixed 15: 4 / 10 (15 words); single sentences: 9 / 10 (about 12 words). (b) “What time does the morning baker start?”: “We are hiring a morning baker.” and “The shift starts at 4:00, ...” are separate sentences, so the chunk that matches the question does not contain the time. A sentence + neighbours chunk contains both. The output also lists each miss with the chunk that was ranked first.